

UNIT 4

LEETCODE EXERCISE

1.Delete Duplicate Emails

Problem Summary

Table: Person

Column Name	Type
id	int
email	varchar

Id is the primary key (column with unique values) for this table.

Each row of this table contains an email.

The emails will not contain uppercase letters.

Task

Write a solution to delete all duplicate emails, keeping only one unique email with the smallest id.

Solution:

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 delete p1
3 from Person p1, Person p2
4 where p1.Email = p2.Email and p1.Id > p2.Id
```

✓ Testcase | > Test Result

Accepted Runtime: 63 ms

✓ Case 1

Input

Person =

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

Output

id	email
1	john@example.com
2	bob@example.com

Expected

id	email
1	john@example.com
2	bob@example.com

Explanation:

john@example.com is repeated two times. We keep the row with the smallest Id = 1.

2. Customers Who Never Order

Problem Summary

Suppose that a website contains two tables, the Customers table and the Orders table.

Table: Customers.

Id	Name
1	Joe
2	Henry
3	Sam
4	Max

Table: Orders.

Id	CustomerId
1	3
2	1

Using the above tables as example, return the following:

Customers
Henry
Max

Task

Write a SQL query to find all customers who never order anything.

Solution:

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select c.Name as Customers
3 from Customers c
4 where c.Id not in (select CustomerId from Orders);
```

✓ Testcase | > Test Result

Accepted Runtime: 94 ms

✓ Case 1

Input

Customers =

id	name
1	Joe
2	Henry
3	Sam
4	Max

Orders =

id	customerId
1	3
2	1

Explanation:

- This query finds customers whose Id is not present in the Orders table.
- A subquery gets all CustomerId values from Orders, and NOT IN excludes those customers.
- Therefore, Henry and Max are returned.

3. Biggest Single Number

Problem Summary

Table my_numbers contains many numbers in column num including duplicated ones.

Can you write a SQL query to find the biggest number, which only appears once.

Note:

If there is no such number, just output null.

Task

Write a SQL query to find the biggest number that appears only once in the my_numbers table.

Solution:

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT MAX(num) AS num
3 FROM (
4     SELECT num
5     FROM MyNumbers
6     GROUP BY num
7     HAVING COUNT(num) = 1
8 ) AS t;
```

☒ Testcase |  Test Result

Accepted Runtime: 128 ms

☒ Case 1

☒ Case 2

Input

MyNumbers =

num
8
8
3
3
1
4
5
6

Output

```
| num |  
| --- |  
| 6   |
```

Expected

```
| num |  
| --- |  
| 6   |
```

Explanation:

- GROUP BY num groups the same numbers together.
- HAVING COUNT(num) = 1 selects only numbers that appear exactly once.
- MAX(num) finds the biggest number among those unique numbers.
- If no number appears once, MAX() returns NULL.

4. Game Play Analysis I

Problem Summary

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

This table shows the activity of players of some game.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on some day using some device.

Task

Write an SQL query that reports the first login date for each player.

Solution:

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select a.player_id, min(a.event_date) as first_login
3 from Activity a
4 group by a.player_id
```

 Testcase | [Test Result](#)

Accepted Runtime: 76 ms

 Case 1

Input

Activity =

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output

player_id	first_login
1	2016-03-01
2	2017-06-25
3	2016-03-02

Expected

```
| player_id | first_login |
| ----- | -
```

```
| 1         | 2016-03-01 |
| 2         | 2017-06-25 |
| 3         | 2016-03-02 |
```

Explanation:

- MIN(event_date) finds the earliest login date for each player.
- GROUP BY player_id groups all login records belonging to the same player.

5. Exchange Seats

Problem Summary

Table: Seat

Column Name	Type
id	int
student	varchar

id is the primary key (unique value) column for this table.

Each row of this table indicates the name and the ID of a student.

The ID sequence always starts from 1 and increments continuously.

Task

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.

Solution

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     a.id id,
4     COALESCE(b.student, a.student) student
5 FROM
6     seat a LEFT JOIN seat b
7     ON b.id = (CASE
8         WHEN a.id%2=1 THEN a.id+1
9         ELSE a.id-1
10        END)
11 ORDER BY a.id
```

 Testcase | [Test Result](#)

Accepted Runtime: 63 ms

 Case 1

Input

Seat =

id	student
1	Abbot
2	Doris
3	Emerson
4	Green
5	Jeames

Output

```
| id | student |
| -- | - - - - |
| 1  | Doris   |
| 2  | Abbot   |
| 3  | Green   |
| 4  | Emerson |
| 5  | Jeames  |
```

Expected

```
| id | student |
| -- | - - - - |
| 1  | Doris   |
| 2  | Abbot   |
| 3  | Green   |
| 4  | Emerson |
| 5  | Jeames  |
```

Explanation:

Note that if the number of students is odd, there is no need to change the last one's seat.

6. Consecutive Numbers

Problem Summary

Table: Logs

Column Name	Type
id	int
num	varchar

In SQL, id is the primary key for this table.
id is an autoincrement column starting from 1.

Task

Find all numbers that appear at least three times consecutively.

Solution

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2
3 SELECT DISTINCT L1.NUM ConsecutiveNums
4 FROM Logs l1, Logs l2, LOGS L3
5 WHERE l1.Id = l2.Id - 1
6       AND L2.ID=L3.ID - 1
7       AND l1.Num = l2.Num
8       AND L2.NUM = L3.NUM;
```

 Testcase | [Test Result](#)

Accepted Runtime: 69 ms

 Case 1

Input

Logs =

id	num
1	1
2	1
3	1
4	2
5	1
6	2
7	2

Output

ConsecutiveNums
1

Expected

```
| ConsecutiveNums |  
| ----- |  
| 1 |
```

Explanation:

1 is the only number that appears consecutively for at least three times.

7. Tree Node

Problem Summary

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table.

Each row of this table contains information about the id of a node and the id of its parent node in a tree.

The given structure is always a valid tree.

Each node in the tree can be one of three types:

"Leaf": if the node is a leaf node.

"Root": if the node is the root of the tree.

"Inner": If the node is neither a leaf node nor a root node.

Task

Write a solution to report the type of each node in the tree.

Solution

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     id,
4     (CASE
5         WHEN p_id IS NULL THEN "Root"
6         WHEN id IN (SELECT DISTINCT p_id FROM tree) THEN "Inner"
7         ELSE "Leaf"
8     END) Type
9 FROM tree;
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 87 ms

☒ Case 1

☒ Case 2

Input

Tree =

id	p_id
1	null
2	1
3	1
4	2
5	2

Output

id	Type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf

Expected

id	type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf

Explanation:

- Node 1 is the root node because its parent node is null and it has child nodes 2 and 3.
- Node 2 is an inner node because it has parent node 1 and child node 4 and 5.
- Nodes 3, 4, and 5 are leaf nodes because they have parent nodes and they do not have child nodes.